

Relatório da Atividade 2 – Etapa 4: Experimentos de Manipulação de Mensagens CAN e Análise de Impacto

Álvaro C. Negromonte¹, Victoria Pessoa Barbosa Matos¹, Pedro C. Guimarães Rodrigues

¹Centro de Informática – Universidade Federal de Pernambuco (CIn/UFPE)

IF747 – Redes Automotivas – 2026.1

Recife – PE – Brasil

[acn3, vpbm, pcgr]@cin.ufpe.br

Resumo. Este relatório apresenta os experimentos de manipulação de mensagens CAN realizados no ambiente Yes, CARLA CAN. Foram executados ataques de *spoofing* sobre duas funções veiculares, freio de mão e marcha ré, além do ataque *fuzzy*. A análise mostra aumento expressivo na quantidade de mensagens do ID do freio de mão e evidencia impactos veiculares observados no CARLA, registrados em vídeo durante a execução dos testes.

1 Introdução

A Etapa 4 da atividade tem como objetivo manipular mensagens CAN e observar o impacto dessas alterações no comportamento do veículo simulado. Diferentemente da etapa de captura e comparação geral do tráfego, esta etapa enfatiza o efeito funcional de ataques específicos sobre funções veiculares.

Foram considerados dois ataques de *spoofing*: um sobre o freio de mão (`hand_brake`) e outro sobre a marcha ré (`reverse`). Também foi executado o ataque *fuzzy*, que injeta mensagens válidas aleatórias para explorar efeitos em diferentes funções veiculares.

2 Configuração Experimental

Os experimentos foram realizados com o ambiente Yes, CARLA CAN, utilizando o barramento virtual `vcan0`. Para esta etapa, foi usado o DBC padrão `data/carla.dbc`, pois o módulo de ciberataques define os IDs CAN a partir do arquivo `attacks/reverse_engineering.p`.

O comando de inicialização recomendado para esta etapa é:

```
./1_up_environment.sh --dbc data/carla.dbc
```

3 Ataques Realizados

3.1 Spoofing do Freio de Mão

O primeiro ataque foi executado sobre a função `hand_brake`. O alvo foi o ID CAN `0x604`, com envio de mensagens periódicas contendo o payload `01`. O comando utilizado foi:

```
conda run --no-capture-output -n n4s_env python3 cyberattacks_module.py
--feature hand_brake --period 0.05
```

Durante o ataque, observou-se que o veículo ficou impedido de se mover normalmente, pois o freio de mão era continuamente forçado pelas mensagens injetadas.

Esse impacto foi registrado em vídeo. Em comparação com o cenário normal, no qual o veículo respondia aos comandos de aceleração, durante o ataque o veículo permanecia parado ou apresentava grande dificuldade para iniciar o deslocamento.

3.2 Spoofing de Marcha Ré

O segundo ataque foi executado sobre a função `reverse`. Durante os testes, foi identificado que o mapeamento original do ataque estava incorreto: o módulo injetava mensagens no ID `0x607` com payload `FF FF FF FF`. No DBC padrão, entretanto, o ID `0x607` corresponde à mensagem `GEAR`, enquanto a mensagem `REVERSE` corresponde ao ID `0x603`.

Por isso, o ataque foi corrigido no arquivo `attacks/reverse_engineering.py`, alterando o alvo para `0x603` e o payload para `01`, compatível com a mensagem de 1 byte definida no DBC:

```
"reverse": {
    "id": 0x603,
    "payload": [0x01]
}
```

Após essa correção, o comando utilizado foi:

```
conda run --no-capture-output -n n4s_env python3 cyberattacks_module.py
--feature reverse --period 0.05
```

Com o ID corrigido, o ataque passou a acionar a indicação de marcha ré no simulador. Esse resultado confirma que o problema observado inicialmente não estava na recepção do CARLA, mas sim no ID CAN usado pelo módulo de ataque.

3.3 Ataque Fuzzy

O ataque *fuzzy* foi executado com o comando:

```
conda run --no-capture-output -n n4s_env python3 cyberattacks_module.py
--feature fuzzy --period 0.05
```

Na implementação atual do repositório, o argumento `-period` não controla o intervalo real do *fuzzy*; o código escolhe funções aleatórias e envia mensagens em intervalos aleatórios entre 0,5 s e 1,0 s. Durante sua execução, foram observados efeitos visuais no simulador. Os efeitos mais fáceis de perceber foram a abertura das portas e o acionamento do freio de mão. Outros efeitos podem ocorrer, mas são mais rápidos ou menos evidentes visualmente, pois as mensagens normais do veículo podem sobrescrever rapidamente parte das mensagens injetadas.

4 Resultados Quantitativos

A Tabela 1 resume os ataques executados e seus efeitos observados.

Tabela 1: Resumo dos ataques executados na Etapa 4.

Ataque	ID CAN	Período	Efeito observado
hand_brake	0x604	0,05 s	Freio de mão forçado
reverse	0x603	0,05 s	Indicador de marcha ré acionado
fuzzy	variável	aleatório	Portas e freio de mão perceptíveis

Para o ataque `hand_brake`, também foi realizada análise quantitativa do tráfego CAN. A contagem do ID 0x604 aumentou de 342 mensagens no cenário normal para 2003 mensagens durante o ataque, totalizando 1661 mensagens adicionais. A taxa do ID atacado passou de aproximadamente 4,82 mensagens/s para 22,78 mensagens/s.

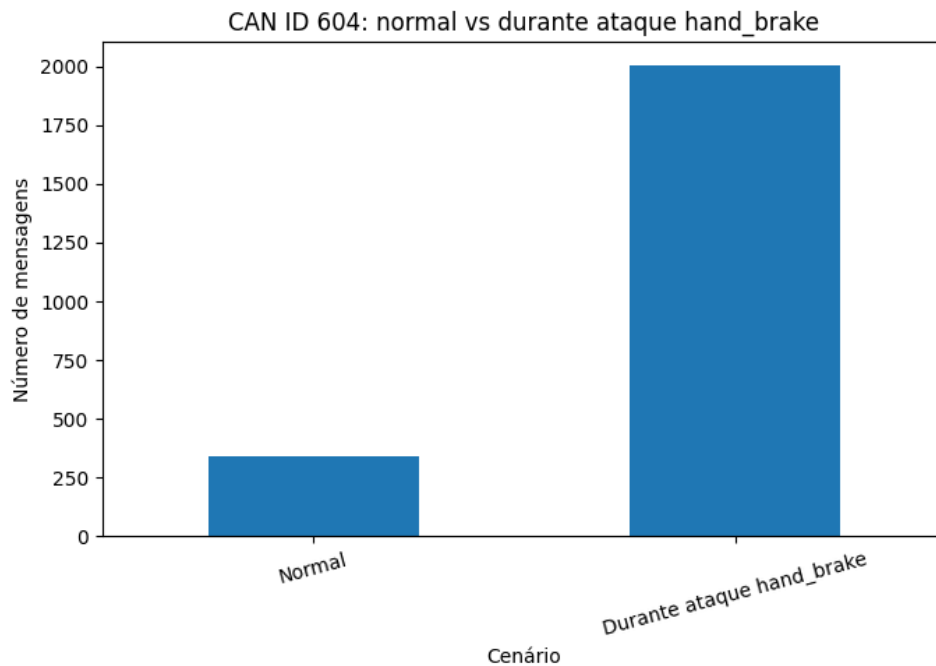


Figura 1: Contagem do ID 0x604 no cenário normal e durante o ataque `hand_brake`.

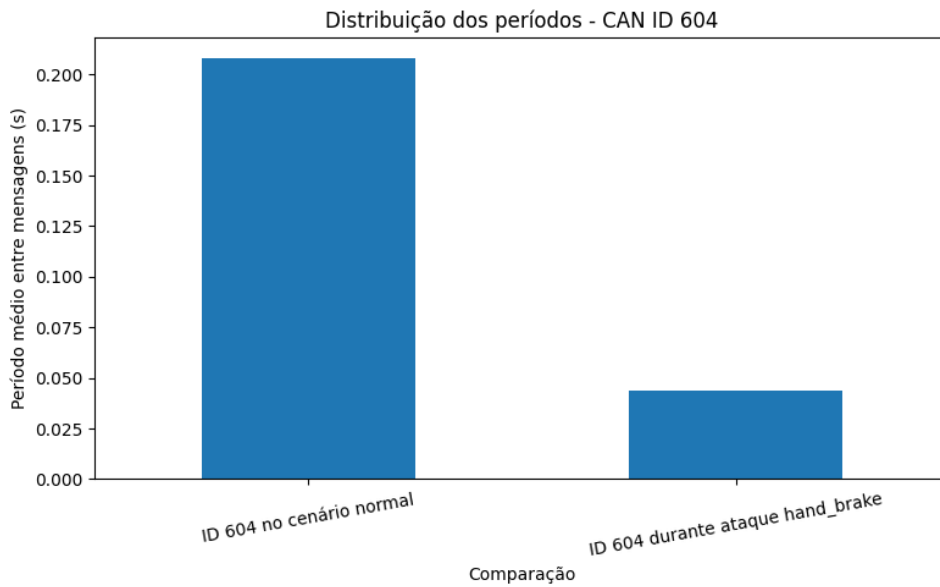


Figura 2: Período médio do ID 0x604 antes e durante o ataque `hand_brake`.

5 Discussão

Os resultados mostram que os ataques de *spoofing* alteraram diretamente o padrão temporal dos IDs atacados quando as mensagens são injetadas com o ID e o payload corretos. O ID 0x604, associado ao freio de mão, passou a aparecer com frequência muito maior durante o ataque, o que explica o impacto observado no simulador: o veículo não conseguia se deslocar normalmente.

O caso do `reverse` também mostra a importância do DBC na interpretação das mensagens. Antes da correção, o ataque enviava mensagens para 0x607, mas esse ID era decodificado como GEAR. Por isso, o campo visual de marcha ré não era acionado. Após alterar o ataque para 0x603#01, o receptor passou a interpretar corretamente a mensagem como REVERSE.

No ataque *fuzzy*, os efeitos observados foram intermitentes, como esperado, pois a função atacada é escolhida aleatoriamente. Mesmo assim, foi possível notar impactos visuais claros quando as mensagens sorteadas afetavam portas ou freio de mão, que são funções de percepção imediata no simulador.

Esse comportamento evidencia uma fragilidade do protocolo CAN: os receptores interpretam mensagens com base no ID e no formato esperado, mas o protocolo não oferece autenticação nativa da origem da mensagem. Assim, uma mensagem falsa mas bem formatada pode ser aceita como legítima pela rede.

6 Conclusão

A Etapa 4 demonstrou que ataques de *spoofing* podem causar impacto funcional visível no veículo simulado e, ao mesmo tempo, gerar alterações mensuráveis no tráfego CAN. O ataque sobre `hand_brake` aumentou expressivamente a contagem do ID 0x604 e impediu o deslocamento normal do veículo. O ataque `reverse`, inicialmente incompa-

tível com o DBC por usar o ID de GEAR, passou a funcionar após a correção para o ID 0x603. O ataque *fuzzy* também produziu efeitos observáveis, principalmente abertura de portas e acionamento do freio de mão.

Os testes foram registrados em vídeo para documentar a diferença de comportamento do veículo com e sem os ataques em execução. Os resultados evidenciam que a injeção de mensagens CAN bem formatadas pode alterar funções veiculares no simulador, reforçando a importância de mecanismos de autenticação, validação e detecção de intrusões em redes automotivas.